Format Data for COLONY

John D. Robinson

2025-09-24

Contents

Format SNP Data for COLONY **1**

Output dataset in 012 format 1

Create COLONY input file 2

Format SNP Data for COLONY

The COLONY run will not be done as part of a markdown (see `runCOLONY.sh`). Here, we'll build the input data file for the COLONY analysis, using functions modified sligthly from those associated with Ellie Weise's sea lamprey project (see Ellie's GitHub repo Nbdlamprey2).

Output dataset in 012 format

We start by generating a file in 012 format, where each genotype in the dataset is coded as a 0, 1, or 2 (based on the number of alternate allele copies present). This can be useful for calculating various measures of diversity and will serve as the basis for our conversion to the format for COLONY below.

n.b., in this format, -1 indicates a missing genotype for an individual at that locus.

```
vcftools --vcf ../snpfiltering/finalSCR.recode.vcf --out finalSCR --012
```

```
VCFtools - 0.1.17
(C) Adam Auton and Anthony Marcketta 2009

Parameters as interpreted:
    --vcf ../snpfiltering/finalSCR.recode.vcf
    --012
    --out finalSCR

Warning: Expected at least 2 parts in INFO entry: ID=AF,Number=A,Type=Float,Description="Estimated allel
Warning: Expected at least 2 parts in INFO entry: ID=PRO,Number=1,Type=Float,Description="Reference all
Warning: Expected at least 2 parts in INFO entry: ID=PAO,Number=A,Type=Float,Description="Alternate all
Warning: Expected at least 2 parts in INFO entry: ID=SRP,Number=1,Type=Float,Description="Strand balanc
Warning: Expected at least 2 parts in INFO entry: ID=SAP,Number=A,Type=Float,Description="Strand balanc
Warning: Expected at least 2 parts in INFO entry: ID=AB,Number=A,Type=Float,Description="Allele balance
```

```
Warning: Expected at least 2 parts in INFO entry: ID=ABP,Number=A,Type=Float,Description="Allele balanc
Warning: Expected at least 2 parts in INFO entry: ID=RPP,Number=A,Type=Float,Description="Read Placemen
Warning: Expected at least 2 parts in INFO entry: ID=RPPR,Number=1,Type=Float,Description="Read Placeme
Warning: Expected at least 2 parts in INFO entry: ID=EPP,Number=A,Type=Float,Description="End Placement
Warning: Expected at least 2 parts in INFO entry: ID=EPPR,Number=1,Type=Float,Description="End Placemen
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=TYPE,Number=A,Type=String,Description="The type of
Warning: Expected at least 2 parts in INFO entry: ID=CIGAR,Number=A,Type=String,Description="The extend
After filtering, kept 1724 out of 1724 Individuals
Writing 012 matrix files ... Done.
After filtering, kept 1325 out of a possible 1325 Sites
Run Time = 0.00 seconds
```

Now read the data into R…

```r
geno <- read.table("finalSCR.012", header = FALSE, row.names=1)
dim(geno) #rows are individuals, columns are loci
```

```
## [1] 1724 1325
```

```r
geno[1:5,1:5]
```

```
##   V2 V3 V4 V5 V6
## 0  0  0 -1  0  0
## 1 -1  1 -1  1  0
## 2  0  0  1  1  1
## 3  0  0 -1  2  0
## 4  1  1 -1  1  2
```

Create COLONY input file

The COLONY input file can be built in a text editor, but it requires quite a bit of information. It may be best to start from a template. Here, we'll build one using a modified function from a former student, Ellie Weise. You can find the `colonydat_create.R` script on Ellie's GitHub repo NbdLamprey2.

First, we need to get the data in a format that COLONY will recognize. Keep in mind that the COLONY input file expects individuals in rows and loci in columns, while our VCF file has the opposite (individuals are in columns). However, the 012 output from `VCFtools` transposes the dataset. Our file has 1724 rows and 1325 columns, so it's in the correct orientation for COLONY already.

COLONY expects alleles to be coded as integers starting with 1. Missing data are coded as 0. The genotypes are expected in two-column format. I'm sure there's a faster way to do this than a big loop over individuals and loci, but since we only need to do it once, speed isn't a huge concern.

```r
#Working with the geno object read in above
nind <- dim(geno)[1] #individuals
nloc <- dim(geno)[2] #loci
#Reading in the individual IDs as well
indv012 <- read.table("finalSCR.012.indv", header = FALSE)
```

```r
#Expanding to two-column format
a1col <- seq(1,nloc*2,by=2)
a2col <- seq(2,nloc*2,by=2)
outmat <- matrix(data = NA, nrow = nind, ncol=2*nloc)
T1 <- Sys.time()
for(i in 1:nind) {
  for(l in 1:nloc) {
    if(geno[i,l] == -1) {
      outmat[i,a1col[l]] <- 0
      outmat[i,a2col[l]] <- 0
    } else if(geno[i,l] == 0) {
      outmat[i,a1col[l]] <- 1
      outmat[i,a2col[l]] <- 1
    } else if(geno[i,l] == 1) {
      outmat[i,a1col[l]] <- 1
      outmat[i,a2col[l]] <- 2
    } else if(geno[i,l] == 2) {
      outmat[i,a1col[l]] <- 2
      outmat[i,a2col[l]] <- 2
    }
  }
}
Sys.time()-T1 #Slow, but not prohibitively
```

```
## Time difference of 53.87191 secs
```

```r
outmat <- cbind(indv012$V1, outmat)
outmat[1:5,1:11] #First 5 individuals, first 5 loci
```

```
##      [,1]                   [,2] [,3] [,4] [,5] [,6] [,7] [,8] [,9] [,10] [,11]
## [1,] "UGA_173101_P020_WD02" "1"  "1"  "1"  "1"  "0"  "0"  "1"  "1"  "1"   "1"
## [2,] "UGA_173101_P020_WC12" "0"  "0"  "1"  "2"  "0"  "0"  "1"  "2"  "1"   "1"
## [3,] "UGA_173101_P020_WC08" "1"  "1"  "1"  "1"  "1"  "2"  "1"  "2"  "1"   "2"
## [4,] "UGA_173101_P020_WC02" "1"  "1"  "1"  "1"  "0"  "0"  "2"  "2"  "1"   "1"
## [5,] "UGA_173101_P020_WC01" "1"  "2"  "1"  "2"  "0"  "0"  "1"  "2"  "2"   "2"
```

```r
dim(outmat)
```

```
## [1] 1724 2651
```

Now source in the `colonydat_create2()` function and use it to create the input file for `COLONY`.

```r
source("../input/colonydat_create2.R")
colonydat_create2
```

```
## function (proj.name = NA, out.name = NA, moms = NA, dads = NA,
##     kids, mar.pref = "SNP@", mar.typ = "0@", err1 = "0.001@",
##     err2 = "0.001@", update.alfs = 1, spp.type = 2, inbreeding = 1,
##     ploidy = 0, fem.gamy = 0, mal.gamy = 0, clone = 1, sib.scale = 1,
##     sib.prior = 0, known.alfs = 0, run.number = 1, run.length = 2,
```

```
##     monitor = 0, windows.version = 0, full.likelihood = 1, likelihood.precision = 2,
##     prob.mom = 0, prob.dad = 0, output_file = "colony2.dat")
## {
##     options(scipen = 999)
##     my.wd <- proj.name
##     my.wd2 <- out.name
##     noff1 <- nrow(kids)
##     nmoms <- nrow(moms)
##     ndads <- nrow(dads)
##     noff <- paste(noff1, "! Number of offspring in the sample",
##         sep = "\t")
##     nloci <- (ncol(kids) - 1)/2
##     nloci <- paste(nloci, "! Number of loci", sep = "\t")
##     random.seed <- round(runif(n = 1, min = 1, max = 9999))
##     random.seed <- paste(random.seed, "! Seed for random number generator",
##         sep = "\t")
##     update.alfs <- paste(update.alfs, "! Not upate/update allele frequency",
##         sep = "\t")
##     spp.type <- paste(spp.type, "! 2/1=Dioecious/Monoecious",
##         sep = "\t")
##     inbreeding <- paste(inbreeding, "! 0/1=No inbreeding/inbreeding",
##         sep = "\t")
##     ploidy <- paste(ploidy, "! 0/1=Diploid species/HaploDiploid species",
##         sep = "\t")
##     gamy <- paste(paste(mal.gamy, fem.gamy), "! 0/1=Polygamy/Monogamy for males & females",
##         sep = "\t")
##     clone <- paste(clone, "! 0/1=Clone inference =No/Yes", sep = "\t")
##     sib.scale <- paste(sib.scale, "! 0/1=Scale full sibship=No/Yes",
##         sep = "\t")
##     sib.prior <- paste(sib.prior, "! 0/1/2/3=No sibship prior/Weak sibship prior/Medium sibship prio
##         sep = "\t")
##     known.alfs <- paste(known.alfs, "! 0/1=Unknown/Known population allele frequency",
##         sep = "\t")
##     run.number <- paste(run.number, "! Number of runs", sep = "\t")
##     run.length <- paste(run.length, "! Length of run", sep = "\t")
##     monitor <- paste(monitor, "! 0/1=Monitor method by Iterate#/Time in second",
##         sep = "\t")
##     monitor.interval <- paste(10000, "! Monitor interval in Iterate# / in seconds",
##         sep = "\t")
##     windows.version <- paste(windows.version, "! Windows version",
##         sep = "\t")
##     full.likelihood <- paste(full.likelihood, "! Fulllikelihood",
##         sep = "\t")
##     likelihood.precision <- paste(likelihood.precision, "! 1/2/3=low/medium/high Precision for Fullli
##         sep = "\t")
##     if (prob.dad == 0 & prob.mom == 0) {
##         probs <- c("0.0", "0.0")
##     }
##     else {
##         probs <- c(prob.dad, prob.mom)
##     }
##     if (is.na(dads) & is.na(moms)) {
##         npars <- c(0, 0)
##     }
```

```
##     else {
##         npars <- c(nrow(dads), nrow(moms))
##     }
##     my.value <- paste0(0, "\n")
##     my.exc.value <- paste(0, 0, "\n")
##     cat(my.wd, my.wd2, noff, nloci, random.seed, update.alfs,
##         spp.type, inbreeding, ploidy, gamy, clone, sib.scale,
##         sib.prior, known.alfs, run.number, run.length, monitor,
##         monitor.interval, windows.version, full.likelihood, likelihood.precision,
##         file = output_file, sep = "\n", append = F)
##     cat("\n", file = output_file, append = T)
##     cat(mar.pref, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(mar.typ, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(err1, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(err2, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     write.table(x = kids, file = output_file, append = T, quote = F,
##         sep = " ", row.names = F, col.names = F)
##     cat("\n", file = output_file, append = T)
##     cat(probs, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(npars, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     if (!is.na(dads) & !is.na(moms)) {
##         cat("\n", file = output_file, append = T)
##         write.table(x = dads, file = output_file, append = T,
##             quote = F, sep = " ", row.names = F, col.names = F)
##         cat("\n", file = output_file, append = T)
##         write.table(x = moms, file = output_file, append = T,
##             quote = F, sep = " ", row.names = F, col.names = F)
##     }
##     cat("\n", file = output_file, append = T)
##     cat(my.exc.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.exc.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.value, file = output_file, append = T)
##     cat("\n", file = output_file, append = T)
##     cat(my.value, file = output_file, append = T)
## }
```

```r
#Note that Wang recommends monogamy model for the clone anlaysis
# Recaptures would be full sibs, not half-sibs
colonydat_create2(proj.name = "CGA",
                  out.name = "CGA",
                  kids=outmat,
                  fem.gamy=1, mal.gamy=1,
                  output_file="CGA_COLONY.dat")
```

Now this can be analyzed in `COLONY` by calling one of the colony executables and pointing it at this input file. Something like:
`mpirun -np 8 ./colony2p.ifort.impi2018.out IFN: CGA_COLONY.dat`

Here's the `.sh` file we'll use to run the `COLONY` job on the cluster:

```r
writeLines(readLines("7_runCOLONY.sh"))
```

```
## Warning in readLines("7_runCOLONY.sh"): incomplete final line found on
## '7_runCOLONY.sh'
```

```
## #!/bin/bash
## #SBATCH --job-name=runCOLONY
## #SBATCH --partition=batch
## #SBATCH --ntasks=8
## #SBATCH --cpus-per-task=1
## #SBATCH --mem=64gb
## #SBATCH --time=48:00:00
## #SBATCH --output=/scratch/jdrobins/CGA_BlackBears/recaps/runCOLONY.o
## #SBATCH --error=/scratch/jdrobins/CGA_BlackBears/recaps/runCOLONY.o
##
## cd /scratch/jdrobins/
## cp -r /home/jdrobins/colony2_Lnx_25_02_2024 ./
## cd colony2_Lnx_25_02_2024
## cp /scratch/jdrobins/CGA_BlackBears/recaps/CGA_COLONY.dat ./
##
## ml impi/2021.6.0-intel-compilers-2022.1.0
##
## srun -n 8 ./colony2p.ifort.impi2018.out IFN:CGA_COLONY.dat
##
## cp CGA.* /home/jdrobins/CGA_BlackBears/recaps/
## cp CGA.* /scratch/jdrobins/CGA_BlackBears/recaps/
##
## scontrol show job ${SLURM_JOB_ID}
```

We'll run that separately (so there is no markdown created for the `COLONY` run).